

6) Chunking, Embeddings, Retrieval

Last updated by | Mennell, Jacob | 19 May 2026 at 13:26 BST

Chunking, Embeddings, And Retrieval

Purpose

This page explains how the solution represents evidence as chunks, how embeddings are used, and how the retrieval layer combines lexical and semantic signals for downstream AI workflows.

Scope

The retrieval layer is designed to support evidence gathering for extraction and checking tasks. It follows a strict retrieval contract: candidates must be filtered correctly at the data layer before any ranking or fusion logic is applied.

Chunk Representation

The core persisted retrieval unit is the document chunk. The chunk table stores:

- chunk identifier
- document identifier
- claim identifier
- document type
- chunk index
- raw chunk text
- vector embedding
- JSON metadata
- optional chunk context

This design lets the system retrieve at chunk level while still preserving parent document context through metadata enrichment.

What Embeddings Are Used For

Embeddings provide the dense similarity signal for semantic retrieval. Query text is embedded at runtime, while chunk embeddings are generated and stored during ingestion.

This allows the system to perform semantic search over persisted chunk vectors in PostgreSQL using pgvector, rather than recomputing embeddings for stored evidence at request time.

Retrieval Design Principles

The retrieval design follows a few clear principles:

- retrieval scope should be enforced in SQL, not inferred after the fact
- filters should restrict candidate inclusion, not act as late ranking tweaks
- fusion should combine independent signals rather than introduce new candidates
- reranking is an optional second stage, not a replacement for candidate scoping

These principles are especially important in multi-claim and multi-document environments where retrieval correctness matters as much as ranking quality.

Lexical Retrieval

The lexical path uses PostgreSQL full-text search with BM25-style ranking. Query text is converted into a `tsquery` form and matched against `text_chunk` using `to_tsvector` and `websearch_to_tsquery`.

The lexical path is useful when exact or near-exact terminology matters, including:

- policy phrasing
- abbreviations or short domain-specific terms
- literal references to claim concepts or labels

The retrieval code also preserves useful lexical signal in cases where returned matches share equal positive BM25 scores.

Semantic Retrieval

The semantic path uses pgvector distance search over stored chunk embeddings. Results are filtered by claim, document type, and optional document IDs before ranking.

The current implementation also expands certain query forms into multiple semantic probes using `build_semantic_query_variants(...)`. Results across those probes are merged with rank-based fusion rather than naive score comparison.

That matters because semantic scores normalized separately inside different probes are not inherently safe to compare directly. The current approach instead rewards chunks that rank well across multiple semantic variants.

Hybrid Retrieval

The repository includes a hybrid retriever that combines multiple relevance signals:

- semantic similarity
- lexical relevance
- optional header overlap signal
- optional recency bias

Two fusion modes are exposed:

- weighted fusion, which is the default extraction-oriented mode
- reciprocal rank fusion, which is a more rank-robust alternative

Weighted fusion uses configurable contribution weights for embedding similarity, BM25, header overlap, and recency. Reciprocal rank fusion is more rank-centric and avoids dependence on absolute score magnitudes.

Header And Recency Signals

Retrieval is not based only on raw text similarity. The code also computes:

- header overlap boosts based on token overlap between query terms and `header_path`
- optional recency boost based on metadata

These signals are designed to improve prioritization without bypassing the primary retrieval scope guarantees.

Reranking

The retrieval stack supports two optional reranking layers:

- embedding-based reranking, which is relatively cheap and deterministic
- LLM-based reranking, which is more instruction-aware but also more expensive and higher-latency

Reranking works on an overfetched candidate set. The retriever first gathers a bounded number of candidates, then optionally trims them to a smaller result set after reranking.

Reranking therefore improves ordering within an already-scoped candidate set; it does not expand retrieval scope.

Retrieval Flow Diagram



flowchart TD

```

Q[Retrieval Query] --> S1[Apply Scope Filters]
S1 --> LEX[Lexical Search]
S1 --> SEM[Semantic Search]
SEM --> VAR[Semantic Variant Fusion]
LEX --> FUSE[Hybrid Fusion]
VAR --> FUSE
FUSE --> BOOST[Header and Recency Signals]
BOOST --> RERANK[Optional Embedding or LLM Rerank]
RERANK --> OUT[Ranked Evidence Chunks]

```

Why This Matters For AI Tasks

The orchestration layer relies on retrieval as the evidence base for extraction and checks. If retrieval is too narrow, the model misses the answer. If retrieval is too broad or badly scoped, the model receives noisy evidence. The current design aims to balance recall, precision, and strict scoping so downstream AI layers receive a stable evidence set.

Key Limitations And Trade-offs

- Semantic retrieval quality depends heavily on chunk quality from ingestion.
- Lexical retrieval is strong for literal terms but weaker when the evidence is conceptually related rather than explicitly phrased.
- Multi-probe semantic search improves recall but increases retrieval complexity.
- Reranking improves candidate ordering but adds latency and, in the LLM case, additional cost.
- Header and recency signals help prioritisation, but they should not be treated as substitutes for correct retrieval scoping.